

Trabajo Práctico Final Inteligencia Artificial

Agustín Fernández Bergé y Ramiro Gatto

28 de julio de 2025

1. Introducción

La idea principal del proyecto es usar InceptionV3 para poder clasificar los distintos “sellos” (gestos con las manos) del anime de Naruto. Para hacer esto hicimos transfer learning sobre el modelo de InceptionV3 más un fine-tuning de las ultimas capas del modelo.

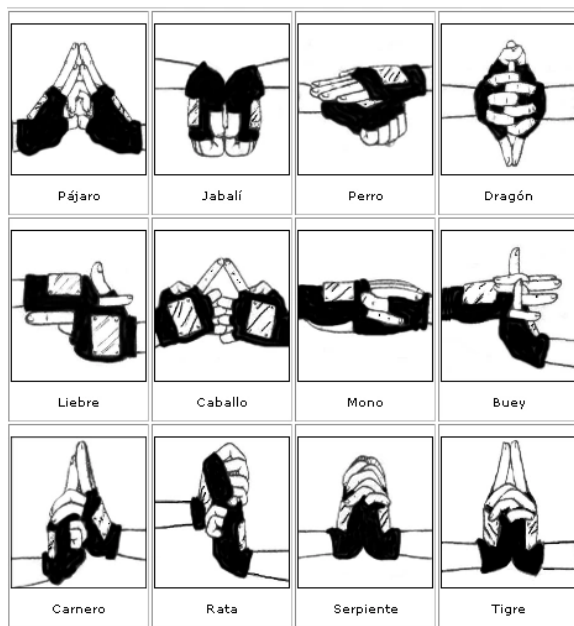


Figura 1: Sellos Básicos

Para el proyecto se utilizaron 3 dataset de proyectos similares (primero empezamos con uno y lo fuimos ampliando), más una cuantas imágenes que agregamos para que este sea un poco más variado.

2. Primer acercamiento

Como experiencia inicial quisimos probar como clasificaba InceptionV3 usando los pesos de Imagenet las imágenes del dataset, para ver en que categorías clasificaba cada sello. AL realizar esto nos dimos cuenta que en algunos sellos una clase se activaba mucho más que las otras, mientras que para otros sellos 3 o 5 clases eran las que mas se activaban respecto a otras. Aquí hay un ejemplo para varios sellos:

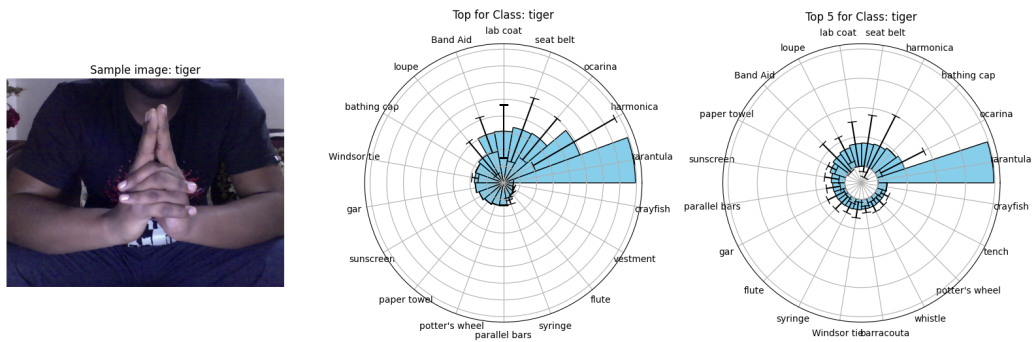


Figura 2: Clasificación de InceptionV3 para el sello del tigre

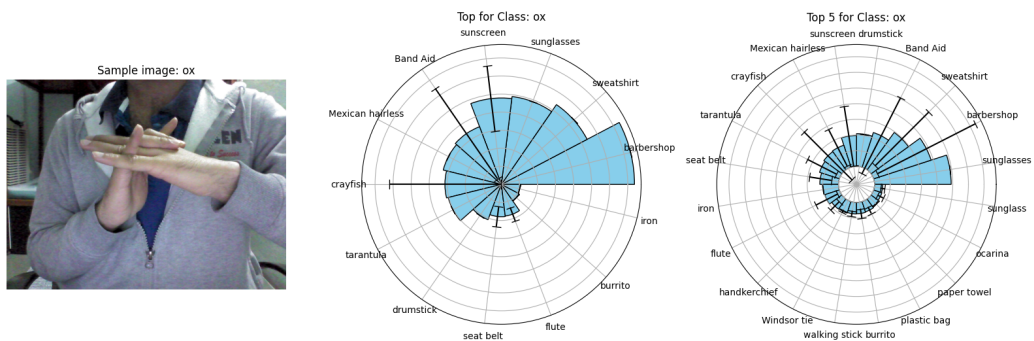


Figura 3: Clasificación de InceptionV3 para el sello del buey

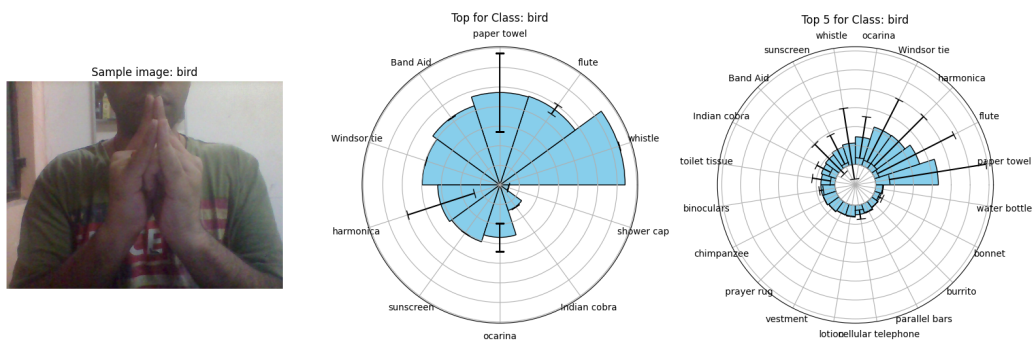


Figura 4: Clasificación de InceptionV3 para el sello del pájaro

Por lo que si queríamos que inception clasificara los sellos, íbamos a necesitar hacer un refinamiento del modelo.

3. Primeros pasos

Lo primero que hicimos fue leer el materia subido en la pagina de Keras sobre transfer learning y fine-tuning [1]. Los pasos de dicho tutorial se pueden resumir como siguen:

1. Cargar los datasets de entrenamiento, validación y test.
2. Aplicar el preprocesamiento correspondiente para que el modelo pueda trabajar con las imágenes. En el caso de InceptionV3, el preprocesamiento consiste en redimensionar las imágenes a un tamaño común y normalizar los valores de los píxeles a valores del conjunto $[-1,1]$.
3. Aplicar data augmentation a los datasets de entrenamiento y validación (este paso inicialmente lo saltamos).
4. Construir las capas del modelo, dejando la capa de InceptionV3 congelada (es decir, sin entrenar) y agregando una capa densa al final para la clasificación.
5. Compilar el modelo, esto es, definir la función de loss, el optimizador y las métricas a utilizar.
6. Entrenar el modelo una cantidad determinada de épocas.
7. Descongelar el modelo de InceptionV3 y realizar un nuevo entrenamiento, esta vez con un learning rate más bajo.
8. Finalmente, evaluar el modelo en los datasets de entrenamiento, validación y test para evaluar rendimiento.

Modificamos un poco las sugerencias del tutorial y agregamos un par de cosas más. Principalmente porque el tutorial esta orientado a clasificar imágenes en 2 clases y nosotros queríamos clasificar en 12 clases. Para esto hicimos lo siguiente:

- En vez de usar la función de loss Binary Crossentropy, usamos Categorical Crossentropy [2]. Estas entropías no son las mismas que se usan en los arboles de decisión, básicamente la Binary Crossentropy se usa para clasificar en 2 clases mientras que la Categorical Crossentropy es una generalización de la Binary Crossentropy para clasificar en más de 2 clases.
- En la capa densa del final del modelo, usamos 12 neuronas (una por cada clase) en lugar de 2 y usamos como función de activación Softmax. Esta ultima es una generalización de la función Sigmoide y es utilizada como función de activación al clasificar en más de 2 clases.

- Cambiamos la métrica del modelo, en cada época se le puede indicar al modelo que calcule una serie de métricas para que las muestre al usuario. Nosotros le indicamos que muestre el accuracy de los conjuntos de entrenamiento y validación en lugar de la binary accuracy.
- Agregamos 3 callbacks para modificar el proceso de entrenamiento:
 - **EarlyStopping**: Si el accuracy del conjunto de validación no mejora en un número determinado de épocas, se detiene el entrenamiento y se restauran los pesos que obtuvieron el mejor accuracy.
 - **ModelCheckpoint**: Si durante el entrenamiento se obtiene una mejora en el accuracy del conjunto de validación, se guardan los pesos del modelo en un archivo en disco.
 - **ReduceLROnPlateau**: Si el accuracy del conjunto de validación no mejora en un número determinado de épocas (menor a las configuradas en **EarlyStopping**), se reduce el learning rate en un factor de 0.1.

3.1. El primer modelo

El primer modelo que hicimos tiene la siguiente estructura:

Layer (type)	Output Shape	Param #	Trainable
rescaling (Rescaling)	(None, 150, 150, 3)	0	-
inception_v3 (Functional)	(None, 3, 3, 2048)	21,802,784	N
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0	-
dropout (Dropout)	(None, 2048)	0	-
dense (Dense)	(None, 13)	26,637	Y

Cuadro 1: Estructura del primer modelo

- **Reescaling** Esta capa se encarga de normalizar los valores de los píxeles de las imágenes a $[-1, 1]$. Esto es necesario para que InceptionV3 pueda trabajar con las imágenes,
- **InceptionV3** La capa base del modelo, inceptionv3 está cargado con los pesos de imagenet y se encuentra congelado, es decir, no se entrena durante el entrenamiento inicial.
- **GlobalAveragePooling2D** Esta capa se encarga de reducir la dimensionalidad de la salida de InceptionV3.

- **Dropout** Esta capa se encarga de apagar, de manera aleatoria, un porcentaje de las neuronas de la capa anterior para evitar el overfitting.
- **Dense** Esta es la capa de salida del modelo, es una capa donde todas las neuronas de la capa anterior están conectadas con 12 neuronas de salida, las cuales tienen una función de activación Softmax.

El primer entrenamiento lo realizamos con la siguiente configuración:

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=init_learning_rate),
    loss='categorical_crossentropy',
    metrics=['accuracy'])
```

- **optimizer** Es el algoritmo que se usa para actualizar los pesos del modelo. En clase y en el tp de redes neuronales nosotros usabamos el optimizador SGD(Stochastic Gradient Descent), Adam presenta muchas mejoras sobre SGD(como learning rate y momentum adaptativo) y es bastante utilizado al entrenar redes neuronales. Con el optimizador tambien configuramos el learning rate inicial, tipicamente probabamos los valores 0.1, 0.01 y 0.001.
- **loss** La función de error que la red neuronal intenta minimizar, como dijimos antes, usamos Categorical Crossentropy.
- **metrics** Las métricas que calcula el modelo en cada época, en este caso usamos accuracy.

Cuando hacemos fine-tuning del modelo, descongelamos una serie de capas de InceptionV3 y entrenamos usando un learning rate más bajo, al inicio descongelabamos las ultimas 20 o 30 capas y usábamos un learning rate de 10^{-5} .

3.2. Primeros problemas

Llegamos a un modelo que posee el siguiente accuracy:

- Accuracy Train: Superior al 80 %
- Accuracy Validation: En torno al 50 %
- Accuracy Test: En torno al 20 %

Donde claramente se ve que ocurre (muy notablemente) overfitting. Además, cuando lo queríamos probar para ver si al menos clasificaba bien los ejemplos del train este los clasificaba casi siempre mal.

3.3. Primera Solución

Entonces como primer medida pensamos que el modelo podría clasificar mal las imágenes debido a que estas eran muy poco “claras”, debido a que eran de poca resolución y las manos eran un poco “particulares”. Sumándole el hecho de que hay algunos sellos muy parecidos (ejemplo, el sello del carnero y el sello del tigre), llegamos a una primera conclusión de que estos factores hacían que el modelo no aprendiese bien.

Entonces como primera solución que aplicamos fue ampliar el dataset, con otros dataset, con manos diferentes al que teníamos, más el agregado de imágenes que sacamos de este canal de youtube [3] el cual hace lo gestos con diferentes atuendos, dándonos el agregado de variedad que necesitábamos.

4. Continúan los problemas

A pesar de haber agregado mas ejemplos y que haya mas variedad seguíamos teniendo overfitting. Era mucho menor teniendo los siguiente resultados para el train y test:

- Accuracy Train: En torno al 90 %
- Accuracy Test: En torno al 60 %

Además seguíamos teniendo el problema de que al probar con ejemplos del train, que deberían dar casi siempre bien por el alto accuracy, las clasificaba mal.

4.1. Segunda solución

Pensamos y aplicamos muchas soluciones, esta por separado no fueron muy útiles (mejoraban ligeramente) pero fue cuando las combinamos que se noto un poco mas la mejora; estas fueron:

- Como primer medida pensamos que el learning rate no era demasiado chico (al crear el modelo y en el fine-tuning). Entonces lo redujimos en ambos casos, haciendo que el del fine-tuning sea menor y que se pueda achicar si es necesario al momento de entrenar.
- La subsecuente medida que se nos ocurrió fue cambiar el fine-tuning. Ahora en vez de congelar y descongelar la ultima capa, lo hacemos con mas (probamos primero con 50 y luego con 100, quedándonos con las 100)

- También, pensamos que los ejemplos actuales podían no ser suficientes, así que agregamos data augmentation para mitigar esto (ya no encontramos mas datasets). Aplicamos las siguientes transformaciones:

- Rotaciones de imágenes.
- Traslaciones horizontales y verticales.
- Zooms.
- Flip horizontal.

Para aplicar data augmentation usamos la clase ImageDataGenerator de Keras, la cual nos permite aplicar estas transformaciones de manera aleatoria a las imágenes del dataset en cada época del entrenamiento.

5. Un nuevo problema

5.1. Nada es lo que parece

Con los cambios que explicamos en la sección anterior hubo una mejora en cuanto a los resultados. Ya el overfitting era mucho menos, la diferencia entre los accuracy del train y test era en el mejor de los casos de solo el 10 %.

Pero seguíamos teniendo el problema de que clasificaba mal prácticamente siempre todas las imágenes. En este punto no entendíamos bien los que pasaba al hacer el test/train saca un accuracy alto (en ambos casos era mayor de 70 %) pero al probarlo nosotros siempre calificaba mal (con estos números al probar 5 imágenes random debería clasificar al menos 3 bien pero nunca pasaba esto, casi siempre clasificaba mal las 5).

5.2. Búsqueda del problema

Para intentar ver el error probamos de ver que categorías asignaba el modelo a cada sello del dataset de test y nos dimos cuenta de que en algunos sellos era cercano al 100 % (es decir, casi siempre los clasificaba bien) y en otros era muy bajo o casi 0. Esto bajaba mucho el accuracy del conjunto de test.

En un inicio lo que pensamos es que el modelo confundía sellos que serían parecidos (como puede ser los sellos de “rat”, “ram”, “tiger” o los de “snake”, “boar”). Así que lo que hicimos fue reentrenar el modelo pero con un grupo reducido de clases

similares. Por ejemplo, usando solo las clases “rat”, “ram”, “tiger”.

Para nuestra desgracia el problema no era por ahí, al juntar las clases parecidas el modelo anda mucho mejor. Es decir con valores altos de los accuracies y no fallando al probarlo con ejemplos del dataset y con ejemplos por fuera de este.

Una solución que nos dieron fue dividir el problema en 2 partes, primero dividir las 12 clases que teníamos en grupos reducidos de 3 o 4 clases, entrenar un modelo que pueda detectar a cual de estos grupos pertenece esta clase y luego entrenar un modelo para cada grupo que pueda clasificar las clases de ese conjunto.

El problema de este enfoque es el tiempo, hay 91 formas de dividir los sellos en grupos de 3; si cada modelo tarda 30 minutos en entrenarse estaríamos cerca de 100 días solo probando que combinaciones de sellos funcionan mejor. Incluso si manualmente eligiéramos los grupos tendríamos que entrenar un total de 5 modelos (el modelo que clasifica los grupos y 4 modelos para cada grupo de sellos) este gran modelo sería bastante grande y podría tardar mucho en clasificar una única imagen. Por lo que antes de intentar esto probamos de buscar otras soluciones.

5.3. Solución que aplicamos

Claramente el modelo no tenía problema en aprender las clases si las aislamos en grupos de 3 o 4, sin embargo al usar muchas clases tendía a favorecer a unas por sobre otras. Así que decidimos investigar si existe alguna solución para este problema. Al final encontramos una solución, esta consistía en cambiar la función de loss.

5.3.1. Categorical Focal Crossentropy

Originalmente usábamos **Categorical Crossentropy** [2] como función de error, la cual tiene la siguiente ley:

$$y_i = \begin{cases} 1 & \text{si la clase es la correcta} \\ 0 & \text{en caso contrario} \end{cases}$$

$\hat{y}_i$ = probabilidad de que la clase sea la correcta

$$L(y, \hat{y}) = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

$$\text{CCE} = \sum_{n=1}^N \sum_{i=1}^C L(y_{in}, \hat{y}_{in})$$

Donde N es el número de ejemplos en el batch, C es el número de clases. Esta función de error es mayor si la probabilidad de la clase correcta es baja y menor si la probabilidad de la clase correcta es alta.

Categorical Focal Crossentropy [4] [5] es una generalización de la función de error anterior, esta es la ley de la función:

$$p_t = \begin{cases} \hat{y}_i & \text{si } y_i = 1 \\ 1 - \hat{y}_i & \text{en caso contrario} \end{cases}$$

$$FL(p_t) = \alpha(1 - p_t)^\gamma L(y, \hat{y})$$

$$\text{CFCE} = \sum_{n=1}^N \sum_{i=1}^C FL(p_t)$$

Donde α es un factor de balanceo, el cual puede ser ajustado para darle más peso a ciertas clases por sobre otras, generalmente se define como el inverso de la frecuencia de la clase en el dataset para paliar el problema del desbalanceo de clases. Nosotros no usamos este factor así que dejamos para todas las clases $\alpha = 0,25$.

El parámetro γ es un factor de modulación, la idea del termino $(1 - p_t)^\gamma$ es hacer que las clases que son más difíciles de clasificar tengan un mayor peso en la función de error y aquellas clases que son más fáciles de clasificar tengan un menor peso en el error. En el entrenamiento del modelo usábamos $\gamma = 2$.

Si $\gamma = 0$ y $\alpha = 1$, CFCE es igual a CCE. Creíamos que aplicar esta función de error podría hacer que el modelo se enfoque más en aquellas clases que siempre eran clasificadas mal y que por lo tanto bajaban el test accuracy del modelo.

5.4. Otros cambios

Una cosa que sacamos fue el dropout al momento de crear el modelo. Si bien la idea del dropout es perturbar el modelo en cada iteración de entrenamiento para que puede aprender de otras formas los conceptos, en estas instancias del tp tratábamos de hacer que las curvas de entrenamiento sean mas “suaves” y que no tengan picos. El dropout causaba que las curvas tengan picos aunque bajásemos el learning rate, por lo que decidimos sacarlo.

También agregamos al entrenamiento un nuevo callback que cortara el entrenamiento si el mismo tardaba más de un cierto tiempo (30 minutos tanto para el entrenamiento inicial como el fine-tuning). Si bien esto podría ser similar a lo que hace el EarlyStopping, este callback utiliza el tiempo de entrenamiento como criterio de parada, mientras que EarlyStopping corta si el accuracy no mejora en un número determinado de épocas. Esto fue mas que nada para poder probar un poco más de veces y evitar que la computadora se muera en el proceso.

Luego de realizar estos cambios el modelo tenia un accuracy de test mayor al 90 % y un accuracy de train mayor al 80 %, y ahora al probarlo clasificaba bien los ejemplo cuando los probábamos nosotros.

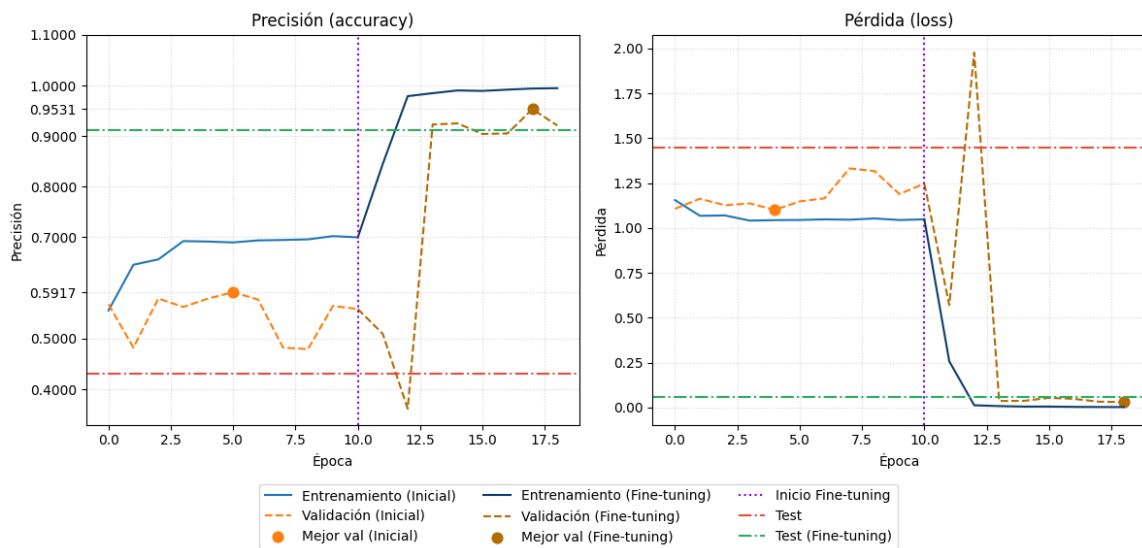


Figura 5: Curvas de entrenamiento obtenidas con el modelo final

Referencias

- [1] Keras (fchollet). Transfer learning and fine-tuning, 2024.
https://keras.io/guides/transfer_learning.
- [2] TensorFlow. Tensorflow: Categorical crossentropy, 2024.
https://www.tensorflow.org/api_docs/python/tf/keras/losses/CategoricalCrossentropy#args.
- [3] K.E.N-DIGIT / Hokage. Canal de youtube sobre los sellos, 2024.
<https://www.youtube.com/@degidoga>.
- [4] TensorFlow. Tensorflow: Categorical focal crossentropy, 2024.
https://www.tensorflow.org/api_docs/python/tf/keras/losses/CategoricalFocalCrossentropy#args.
- [5] Raúl Gómez. Understanding categorical cross-entropy loss, 2024.
https://www.tensorflow.org/api_docs/python/tf/keras/losses/CategoricalFocalCrossentropy#args.